

Bringing the Zarr and OME-Zarr formats to R with the *Rarr* and *romeo* Bioconductor packages

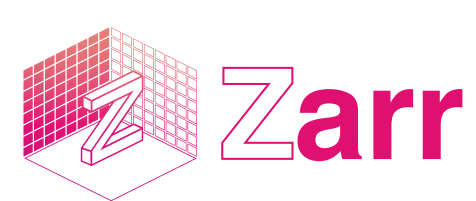
Hugo Gruson¹, Mike Smith¹, Artür Manukyan²

1. EMBL, Heidelberg, Germany

2. MDC, Berlin, Germany

Background: the Zarr and OME-Zarr formats

OME-Zarr is rapidly replacing (OME-)TIFF and other proprietary formats across spatial biology tools.



- chunked, compressed, format for N-dimensional data
- fully open and community-led format, with implementations in multiple programming languages.
- partial & lazy reading, including on remote data stores (e.g., S3)
- good disk space / speed, tunable via compression settings

OME adds metadata conventions for:

- relationships between images in a set
- multiscale image pyramids
- cross-instrument / cross-coordinate system transformations

OME-Zarr = a plain Zarr container, plus additional attributes (.zattrs / zarr.json) describing the relationship between images

Motivation: why develop R packages for these formats?

Goal

A reliable Zarr/OME-Zarr implementation for R to:

- Unlock this data type for existing R packages (*SpatialExperiment*, *Seurat*, *OSCA*, *EBImage*...)
- Improve interoperability between R and other programming languages
- Serve as the foundation for more specialized analysis packages

Scope

- Full reading & writing support for Zarr v2 & v3 + OME-Zarr v0.4 & v0.5.
- Local and S3 storage
- Support for numerous Zarr extensions (compression, data types, etc.). New extensions implemented on a case by case basis by user request.

Design principles

Accessibility

Need to be installable and usable by technical and non-technical users, on any computer (laptop, cluster) and OS.

⇒ The packages can be installed via Bioconductor and provide very detailed documentation, allowing a seamless integration in all kinds of downstream pipelines (*spatialdataR*, *SpatialExperiment*, *Seurat*, etc.)

Reliability

Need to work on the user first try.

⇒ Extensive test suite and continuous integration to ensure we don't accidentally break them in the future.

Maintainability

Must live on for the next decades, even if funding and maintenance capabilities disappear.

⇒ The code is regularly simplified and refactored to ensure complexity remains low.

Performance

Need to work with all data sizes (up to several dozen TB), and even with less powerful machines.

⇒ The speed and memory footprint of reading & writing is minimized.

Extra performance can be obtained by using lazy reading (through the *ZarrArray* package).

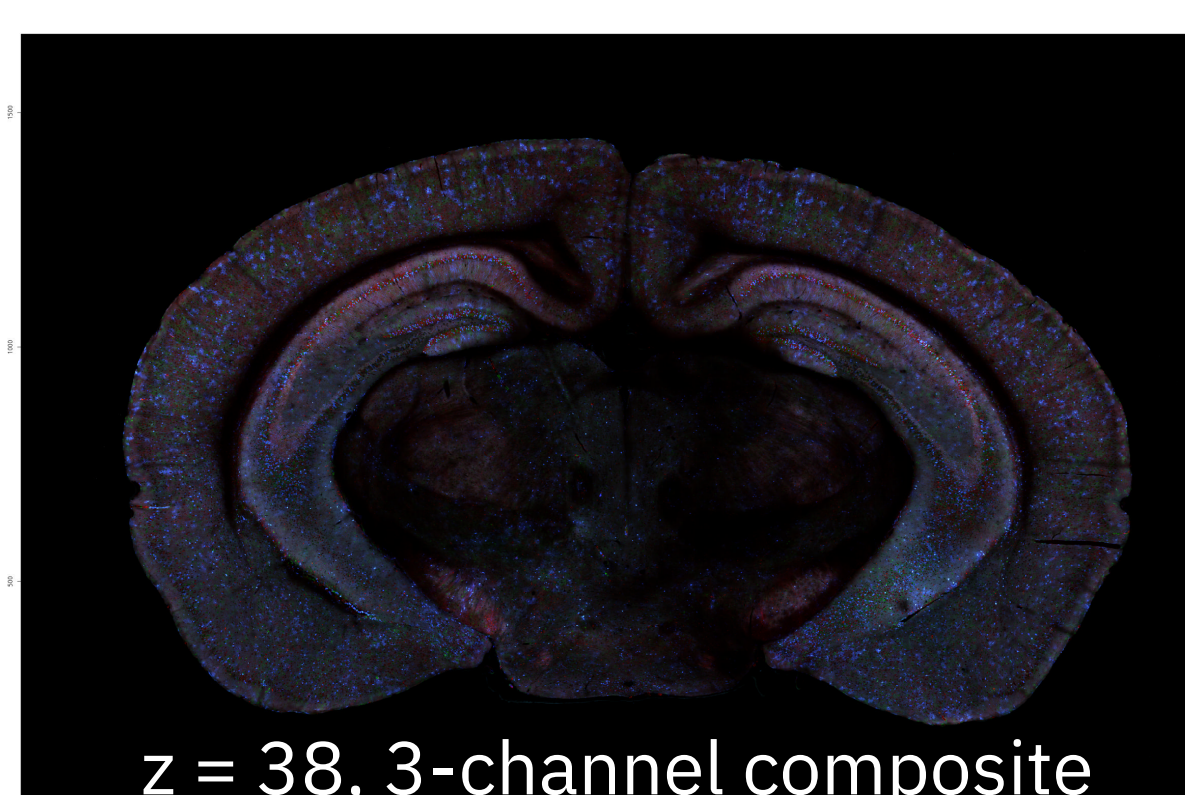
⇒ Continuous benchmarking ensures we only move towards better performance as time progresses

Output & outcome

We can do a partial read of the entire FOV and the 3 channels on a single z slice of the following file:

- Dimensions: 3 channels, 91 z, 6675 y, 9560 x
- Size of the equivalent uncompressed TIFF: 34.8 GiB
- Size on disk: 12.8 GiB

```
Rarr.R
z <- read_zarr_array(
  "https://livingobjects.ebi.ac.uk/idr/zarr/v0.4/idr0048A/9846152.zarr/1",
  index = c(NULL, 38, NULL, NULL)
)
```



With a single CPU thread:

- Time spent: 3 seconds
- Peak memory usage: 1.4 GiB

